

TRABALHO PRÁTICO: EXPLORANDO P4 PARA ANÁLISE DE DESEMPENHO

Carlos Negri — 00333174
Tiago Elias Steinke — 00171884

1 Objetivo

O presente trabalho teve como objetivo projetar e implementar um mecanismo de *In-Band Network Telemetry* (INT) em uma rede programável utilizando a linguagem P4. A solução permite que informações de monitoramento (como atrasos, portas utilizadas, tamanho de pacotes e identificação dos nós) sejam adicionadas dinamicamente aos pacotes durante seu percurso pela rede, sendo posteriormente extraídas por um hospedeiro receptor sem interferir no tráfego de controle da infraestrutura.

2 Arquitetura do Plano de Dados

Para viabilizar a telemetria transparente, foram realizadas modificações no arquivo `basic.p4`, separadas nas seguintes etapas arquiteturais:

2.1 Estrutura dos Cabeçalhos INT

Foram criados dois novos tipos de cabeçalho baseados na especificação INT:

- **INT Master (Cabeçalho Pai - 4 bytes):** Atua como o *shim header*. Armazena o protocolo encapsulado original (para posterior restauração), o contador dinâmico de saltos (`num_slave`) e o comprimento total da área de telemetria (`int_length`).
- **INT Slave (Cabeçalho Filho - 12 bytes):** Inserido por cada switch no caminho. Foi expandido além dos requisitos mínimos para armazenar:
 - ID dinâmico do switch (2 bytes);
 - Porta de entrada (1 byte);
 - Porta de saída (1 byte);
 - Timestamp de entrada no formato global (4 bytes);
 - Tamanho total do pacote na entrada do pipeline (4 bytes).

2.2 Lógica de *Parsing* e *Deparsing*

O *Parser* foi atualizado para inspecionar o campo `protocol` do IPv4. Caso identifique o valor 253 (protocolo customizado para o INT local), ele desvia a extração para o

INT Master e entra em um laço de repetição condicional (*loop*) para extrair múltiplos cabeçalhos INT Slave com base no campo `num_slave`. O *Deparser* emite os cabeçalhos validos na ordem correta antes do *payload*.

2.3 Processamento no Egress (Lista Branca e IDs Dinâmicos)

Afastando-se de abordagens convencionais nossa arquitetura implementou a lógica de encapsulamento estrategicamente no bloco **MyEgress**. Essa decisão permitiu duas inovações essenciais:

1. **Filtro de Lista Branca (Whitelist):** O P4 foi programado para injetar cabeçalhos INT estritamente em pacotes de aplicação baseados em **TCP (6)**, **UDP (17)** ou pacotes que já contenham telemetria (Protocolo 253). Isso garante que ferramentas de controle da rede, como o Ping nativo (ICMP), transitem pela rede com 100% de transparência, sem que seus cabeçalhos sejam adulterados.
2. **Dedução de ID em Tempo de Execução:** Como a arquitetura `v1model` não provê um ID estático nativo para o switch no plano de dados sem a ajuda do plano de controle, desenvolveu-se uma heurística puramente baseada em Data Plane. O switch extrai seu próprio endereço MAC dinamicamente (através de um deslocamento de *bits* $\gg 8$ e uma máscara $\& 0xFF$) para carimbar o cabeçalho *Slave* com o seu identificador real (Ex: Switch 1, 2, 3 ou 4), mantendo a topologia completamente genérica.

3 Arquitetura do Plano de Aplicação

Os *scripts* de transmissão (`send.py`) e recepção (`receive.py`) foram integralmente reescritos para operar como *Dashboards* em interfaces TUI (*Text User Interface*).

- **Transmissor:** Descobre dinamicamente a rede à qual pertence, monta o cabeçalho de enlace (MAC) apontando para o seu *gateway* (Switch) de forma autônoma sem o uso de endereços de *broadcast*, e injeta pacotes TCP portando mensagens em texto plano.
- **Receptor:** Intercepta pacotes com protocolo IPv4 253. Realiza a decodificação da estrutura de bytes em formato *Big-Endian*, separando o cabeçalho Pai, iterando sobre a memória para coletar todos os 12 bytes de cada salto, e reconstruindo a camada final de transporte para extrair o texto em claro. Os dados são dispostos em uma tabela de rastreamento de *hops* atualizada em tempo real.

4 Topologia Expandida e Validação

Para atestar o requisito de que o mecanismo deve funcionar em topologias genéricas independentemente do número de dispositivos, a rede triângulo original foi modificada.

Criou-se um quarto nó (**Switch 4**) conectado em série ao **Switch 3**, permitindo validar saltos mais longos e rotas assimétricas.

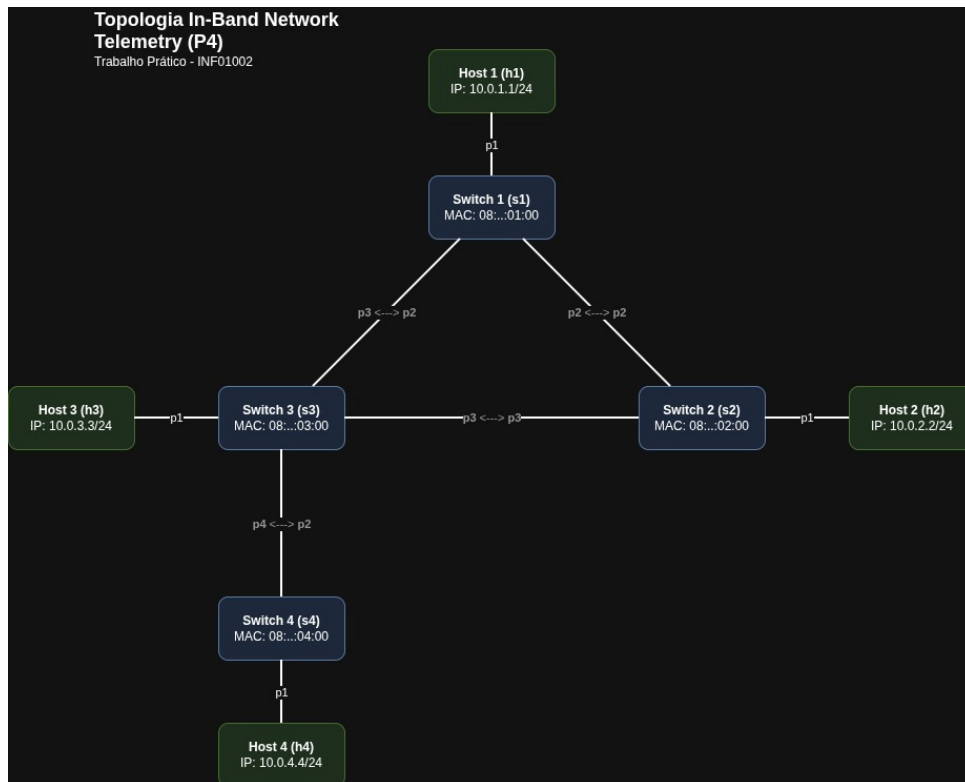


Figure 1: Topologia da rede.

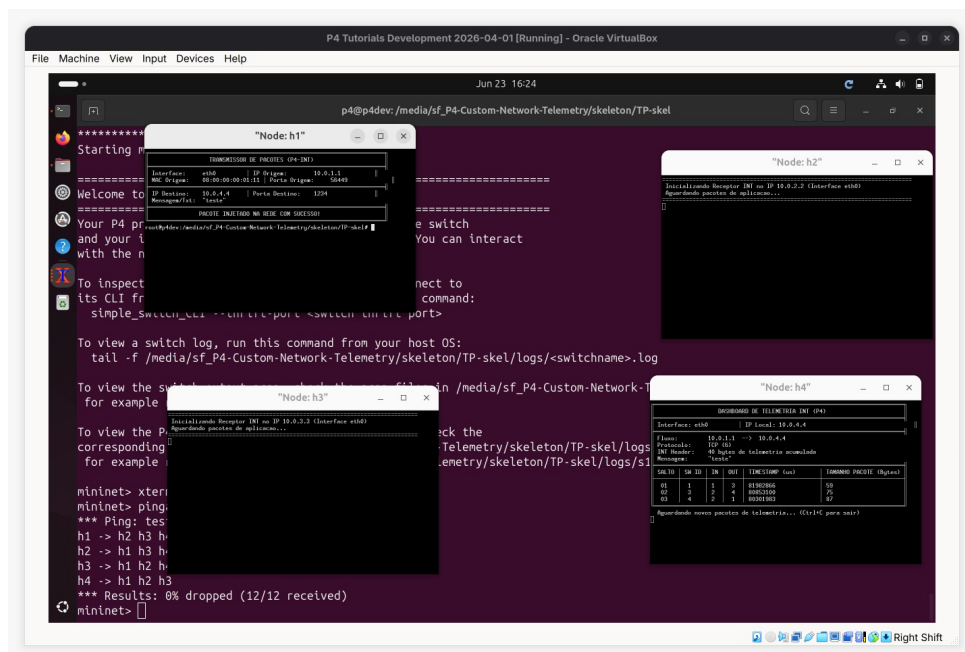


Figure 2: Teste com 3 saltos.

Durante a extração dos dados na tabela TUI, observou-se que o *timestamp* de um switch final no caminho pode apresentar valor inferior ao de um switch anterior. Esse compor-

tamento comprova a natureza da simulação do BMv2 no Mininet, onde os processos não possuem sincronismo via *Precision Time Protocol* (PTP), tornando os relógios independentes. Contudo, o acúmulo de saltos opera de forma contínua e robusta.

5 Conclusão

A implementação da *In-Band Network Telemetry* (INT) em P4 provou ser um desafio técnico altamente enriquecedor, revelando as minúcias e complexidades operacionais inerentes ao plano de dados de redes programáveis.

Durante o desenvolvimento, diversas dificuldades técnicas foram encontradas, sendo a principal delas a interferência inicial da telemetria no tráfego de controle da rede (como os pacotes ICMP utilizados pelo comando `ping`).

Outro desafio significativo foi a atribuição de identificadores aos switches sem violar a regra de manter a topologia genérica e não adulterar o plano de controle (*Control Plane*).

Como ganho educacional, o trabalho consolidou de forma prática os conceitos de *Software-Defined Networking* (SDN) e a separação dos planos de rede. Os conhecimentos sobre o encapsulamento estrutural de protocolos, a manipulação de pacotes byte a byte via código e a interação entre a camada de enlace e de rede foram aprofundados.

Por fim, a integração do plano de dados em P4 com a criação de *dashboards* interativos via *scripts* Python conectou a lógica de baixo nível dos switches BMv2 com a visualização de telemetria em alto nível. Isso proporcionou uma visão moderna de como monitoramento em tempo real.